MiniChat – mesagerie locală cu istoric

Subiectul 44 – Aplicații client-server
Echipa: Popa Ghita Didita Nicola Andra și Duna Bianca Stefania · Grupa 422C
Repository: https://github.com/kommanderm/Proiect_Python_Poli

1. Prezentare generală şi obiective

Titlul proiectului şi echipa

Proiectul se numește **MiniChat** și este realizat de Popa Ghita Didita Nicola Andra și Duna Bianca Stefania (grupa 422C).

Context şi scop

Un grup mic de utilizatori, de exemplu o echipă la laborator, are nevoie de o mesagerie care funcționează complet local: fără conturi în cloud și fără internet, cu un istoric care nu se pierde dacă un client sau serverul se oprește. MiniChat rezolvă această problemă printr-un server Python care rulează pe același laptop cu două sau trei ferestre-client. Utilizatorii țintă sunt utilizatori demonstrativi cu identități fictive, care schimbă mesaje în camere tematice.

Obiective principale

1. Dezvoltarea unui server local care acceptă un mesaj doar de la o sesiune validă, membră a camerei, îi atribuie un ID și un marcaj temporal și respinge textul gol sau mai lung de 1000 de caractere.
2. Salvarea tuturor mesajelor în SQLite și reîncărcarea ultimelor 100 de mesaje la deschiderea unei camere și după reconectare, fără mesaje dublate.
3. Livrarea mesajelor în timp real, fără blocarea interfeței, pentru cel puțin trei clienți simultani.
4. Verificarea automată a tuturor rutelor HTTP și a evenimentelor Socket.IO prin 116 teste, rulate la fiecare pornire a serverului.

Limitele proiectului

- Nu include parole sau autentificare OAuth2: utilizatorul se autentifică cu un ID generat la înregistrare.
- Nu folosește HTTPS; aplicația este destinată doar aceluiași laptop sau aceleiași rețele locale.
- Nu folosește stocare în cloud, ci un fișier SQLite local (`db.sqlite`).
- Nu are roluri de administrator, ștergere sau editare de mesaje, fișiere atașate sau notificări.
- Folosește serverul de dezvoltare Werkzeug, potrivit pentru demonstrație, nu pentru producție.

2. Cerinţe funcţionale

Funcţionalităţi

Must-have (implementate):

1. Înregistrare cu nume afișat; serverul generează un ID de autentificare de 12 caractere.
2. Autentificare cu ID-ul primit și deconectare; sesiunea este păstrată 7 zile.
3. Camere: creare, intrare (*Join*) și părăsire (*Leave*).
4. Lista camerelor împărțită după permisiune: camerele proprii și celelalte camere.
5. Trimiterea și primirea mesajelor în timp real, în camera selectată.
6. Istoric salvat în SQLite; la deschiderea camerei se încarcă ultimele 100 de mesaje.
7. Reconectare automată după oprirea serverului, cu reîncărcarea istoricului.

În lucru (cerințe ale subiectului încă neacoperite complet): un identificator de operație care împiedică dublarea unui mesaj trimis de două ori, reluarea citirii exact după ultimul ID primit, un indicator permanent al stării conexiunii și adnotări de tip în tot codul.

Nice-to-have: contor de mesaje necitite pentru celelalte camere, paginarea istoricului mai vechi de 100 de mesaje, editarea sau ștergerea propriilor mesaje.

Scenarii de utilizare

- **Înregistrare:** utilizatorul introduce un nume afișat -> serverul validează numele și generează ID-ul -> ID-ul este afișat o singură dată -> sesiunea pornește.
- **Autentificare:** utilizatorul introduce ID-ul -> serverul caută utilizatorul -> sesiunea este creată -> se deschide pagina de chat.
- **Trimiterea unui mesaj:** utilizatorul scrie mesajul -> serverul verifică sesiunea, apartenența la cameră și lungimea textului -> mesajul primește un ID și o oră și este salvat în SQLite -> serverul îl trimite membrilor camerei -> fiecare client verifică permisiunea și afișează mesajul.
- **Deconectare și reluare:** serverul se oprește -> clientul afișează „Connection lost, retrying…” și reîncearcă -> la revenire, clientul reintră în cameră și reîncarcă istoricul fără mesaje dublate.

Interfaţa cu utilizatorul

Aplicația are o interfață web minimală, realizată cu Flask, HTML, CSS și JavaScript simplu. Clientul Socket.IO este inclus în proiect, deci nu este nevoie de internet. Există două pagini: pagina de autentificare (formularele *Register* și *Log in*) și pagina de chat (lista camerelor, conversația selectată cu autorul și ora fiecărui mesaj și formularul de trimitere).

Sesiunea este păstrată în browser, deci toate filele aceluiași browser aparțin aceluiași utilizator. Pentru mai mulți utilizatori pe același laptop se folosesc browsere diferite sau ferestre incognito.

3. Arhitectură, tehnologii şi structura datelor

Stiva tehnologică

- Python 3.14;
- Flask 3.1 – server web, rute şi sesiuni;
- Flask-SocketIO 5.7 – mesaje în timp real;
- Flask-SQLAlchemy 3.1 şi SQLAlchemy 2.1 – acces la baza de date (ORM);
- SQLite – baza de date locală;
- Socket.IO 4.8.1 – clientul JavaScript din browser;
- unittest – testele automate;
- uv sau venv + pip – instalarea dependenţelor (`uv.lock`, `requirements.txt`).

Modelul de date

Tabel	Atribute	Rol
user	id (text de 12 caractere, cheie primară), registered_at (data înregistrării), display_name (text, 1–32 caractere)	utilizatorii
room	id (întreg, cheie primară), display_name (text unic, 1–32 caractere)	camerele
user_in_room	user_id (referă user.id), room_id (referă room.id); cheie primară compusă	apartenenţa la camere
message	id (întreg, cheie primară, stabileşte ordinea), room_id (referă room.id), user_id (referă user.id), content (text, 1–1000 caractere), sent_at (ora serverului, UTC)	mesajele

Permisiunea unui utilizator într-o cameră este *member* dacă există rândul corespunzător în `user_in_room`, altfel *none*.

Arhitectura codului

Stocarea, rutele HTTP, evenimentele în timp real şi interfaţa sunt separate în module distincte:

```
minichat/
├── run.py              rulează testele, apoi porneşte serverul
├── deploy.py           instalează dependenţele (uv sau pip) şi porneşte aplicaţia
├── requirements.txt    dependenţele pentru pip
├── app/
│   ├── __init__.py     aplicaţia Flask
│   ├── config.py       configuraţia
│   ├── validation.py   validarea datelor primite
│   ├── db/             modelele şi stocarea (SQLAlchemy)
│   ├── routes/         API-ul HTTP
│   ├── socket/         evenimentele Socket.IO
│   ├── templates/      paginile HTML
│   └── static/         CSS şi JavaScript
└── tests/              testele automate
```

4. Planificare, testare şi livrabile

Etape de implementare

- **Faza 1 – Design şi structura bazei de date:** tabelele și modelele SQLAlchemy. Finalizată.
- **Faza 2 – Implementarea logicii backend:** API-ul HTTP, sesiunile, evenimentele Socket.IO și validările. Finalizată.
- **Faza 3 – Interfaţă şi integrare:** paginile de autentificare și chat, reconectarea automată. Finalizată.
- **Faza 4 – Testare şi documentare:** testele automate, scriptul de instalare și README. Finalizată.
- **Faza 5 – Cerinţele rămase ale subiectului:** identificatorul de operație, reluarea după ultimul ID, indicatorul de conexiune și adnotările de tip. În lucru.

Strategia de testare

Aplicația este verificată cu 116 teste automate scrise cu `unittest`. Testele folosesc o bază SQLite în memorie, deci baza reală nu este modificată. Ele rulează automat la fiecare pornire (`python run.py`); dacă un test eșuează, serverul nu pornește. Testele acoperă:

- **fluxul normal:** un mesaj trimis ajunge la toți membrii camerei și apare identic în istoric;
- **cazuri-limită:** un mesaj de exact 1000 de caractere este acceptat, iar istoricul returnează doar ultimele 100 de mesaje;
- **cazuri de eroare:** mesajele goale, prea lungi sau trimise de utilizatori care nu sunt membri sunt respinse și nu se salvează; datele invalide (ID-uri greșite, caractere de control sau invizibile) sunt respinse sau curățate.

Testele sunt împărțite în cinci fișiere: paginile (8 teste), autentificarea (23), camerele și istoricul (36), evenimentele Socket.IO (31) și validarea datelor (18).

Instalare şi rulare

```
git clone https://github.com/kommanderm/Proiect_Python_Poli.git
cd Proiect_Python_Poli
python3 deploy.py
```

Scriptul folosește uv dacă este instalat, altfel creează un mediu virtual și instalează pachetele cu pip. Apoi rulează testele și pornește serverul pe portul 5000.

Livrabile finale

- repository Git cu codul sursă;
- fișierul README.md cu instrucțiuni de instalare, dependențe, rulare și surse;
- testele automate din directorul `tests/`;
- scriptul de instalare `deploy.py` și fișierul `requirements.txt`;
- această prezentare.